AI Stylist & Wardrobe Manager -- Engineering System Documentation

Multimodal Vision Intake, HITL Refinement, LangGraph Stylist Engine & Telegram Client

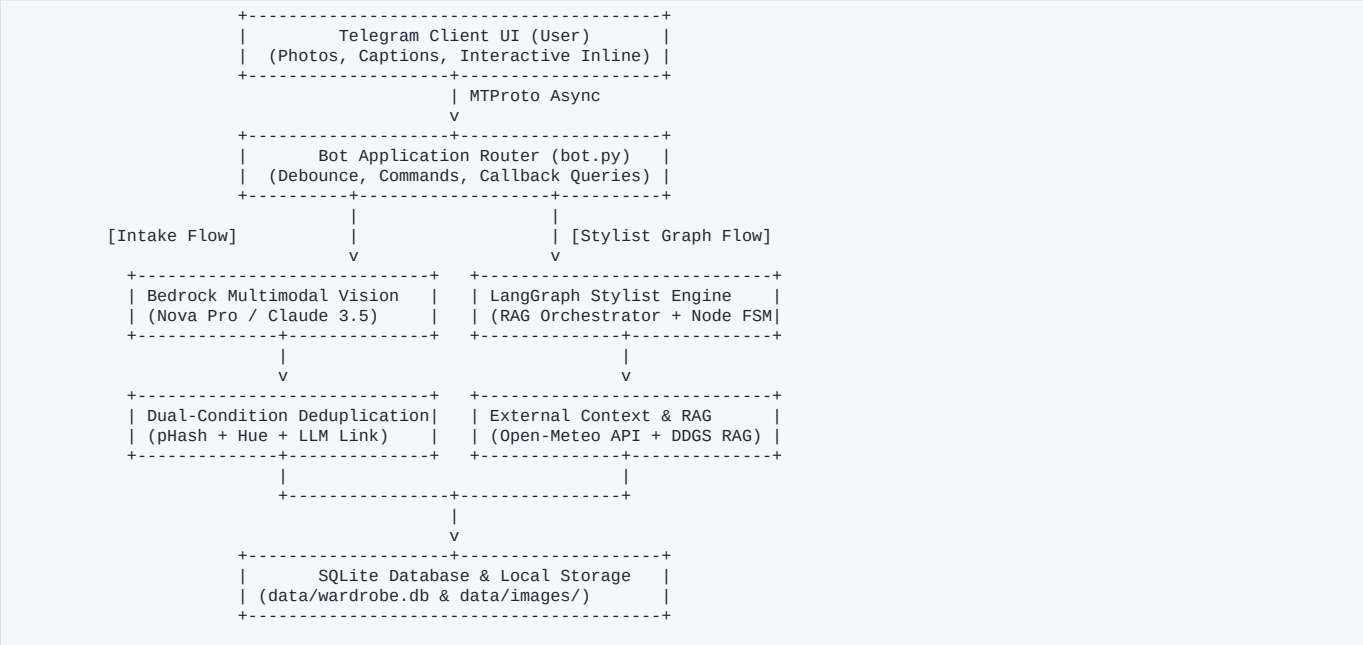
1. Methodology & Functional Overview

The system eliminates the cognitive fatigue of daily outfit selection by transforming unstructured casual wardrobe photos into a structured digital inventory, combining live contextual retrieval (Weather RAG, Web Trend scraping, User History RAG) with a self-critiquing LangGraph agentic loop to generate personalized outfit recommendations.

Core Functional Capabilities:

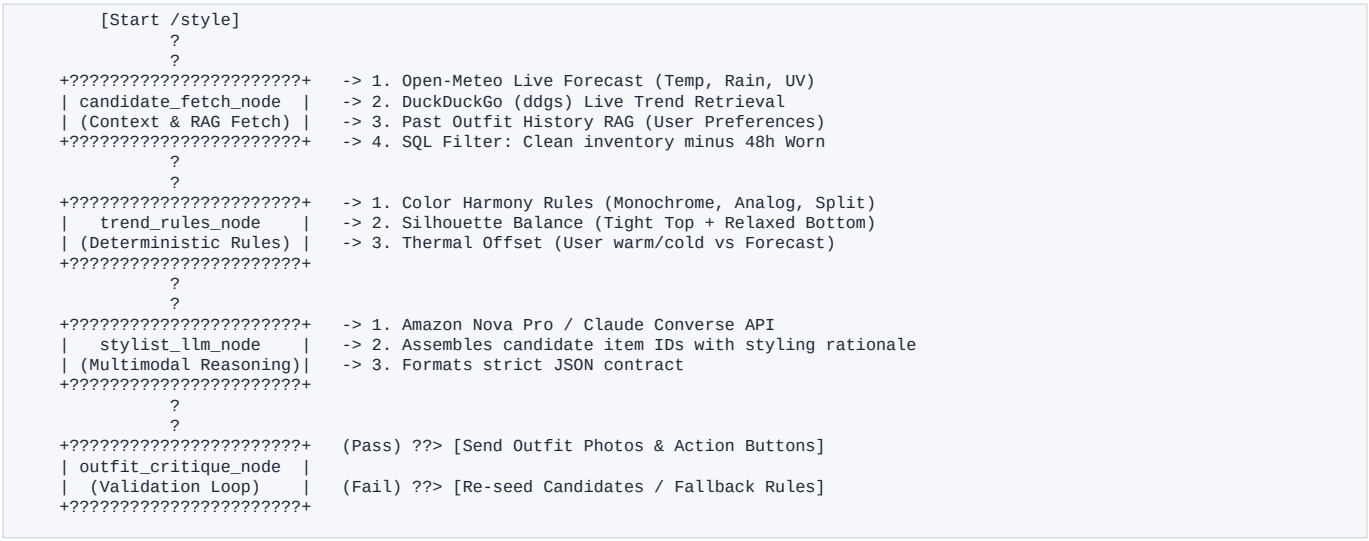
- Multimodal Vision Intake (app/extractor.py): Automatically detects single-garment items or full multi-piece Outfits of the Day (OOTD). Extracts structured Pydantic schemas: category, sub-category, primary/accent colors, silhouette fit, fabric weight, formality tier (1-5), and styling tags.
- Human-In-The-Loop (HITL) Verification (app/handlers.py): Extracted items are staged in an unconfirmed state (is_verified=0). Users verify with 1-tap or refine attributes in conversational language (e.g. 'the shirt is navy linen, brand is Uniqlo'), triggering authoritative re-extraction.
- Conservative Dual-Condition Duplicate Prevention: Imposes a strict gate: candidates must match both category and primary color hue before perceptual hashing (64-bit pHash) and Bedrock LLM identity checks can link them as appearances of existing items.
- Interactive Profile Onboarding (app/profile_flow.py): 7-step Telegram wizard capturing body build, proportions (e.g. long torso, broad shoulders), preferred silhouettes, avoided colors, and thermal preference (runs warm/cold).
- Contextual Agentic Stylist Graph (app/stylist_graph.py): Executes an asynchronous LangGraph pipeline integrating live Open-Meteo weather forecasts, DuckDuckGo (ddgs) web fashion trends, past OOTD history RAG, and a 48h anti-repeat wear cooldown.
- Compact 2-Step Wardrobe & Laundry Management: Organized photo album views with in-place Edit, Delete, and Laundry actions. Word-capped preview badges (4 words) ensure zero UI overflow while SQLite retains full rich descriptions for LLM reasoning.

2. High-Level System Topology



3. LangGraph Stylist Architecture & State Machine

The outfit generation engine is implemented as a stateful, cyclical LangGraph state machine (app/stylist_graph.py). It decouples context gathering, deterministic constraint filtering, LLM reasoning, and validation critique into modular nodes:



Detailed LangGraph Node Responsibilities:

- 1. candidate_fetch_node: Queries Open-Meteo REST API for live weather, executes DuckDuckGo (ddgs) search for dress codes, pulls user outfit history, and retrieves clean inventory excluding garments in laundry or worn within the last 48 hours.
- 2. trend_rules_node: Evaluates deterministic rules from app/style_matrix.py. Checks color compatibility, computes silhouette ratios, and applies thermal offsets based on user profile preferences.
- 3. stylist_llm_node: Prompts AWS Bedrock Converse API with prompt-injected RAG context, profile constraints, and available items to assemble the optimal multi-piece combination.
- 4. outfit_critique_node: Validates that all recommended item IDs exist in clean inventory, checks formality consistency, and triggers an automatic deterministic fallback if LLM constraints fail.

4. Codebase Structure & Module Map

The repository is architected with strict separation of concerns, single-responsibility modules, and comprehensive error boundaries:

File / Module	Layer	Responsibilities & Architectural Role
bot.py	Entrypoint	Initializes DB, configures PTB Application, registers handlers, starts async polling.
app/config.py	Config	Loads .env credentials, parses Settings dataclass, validates AWS & Telegram keys.
app/database.py	Persistence	SQLite connection manager, schema migrations, CRUD queries, wear history logging.
app/extractor.py	Vision / AI	AWS Bedrock Converse API client, vision extraction prompts, 64-bit pHash, badge renderer.
app/handlers.py	Controller	Telegram commands (/wardrobe, /style, /laundry, /delete), intake debouncing, callbacks.
app/models.py	Contracts	Pydantic v2 schemas: ExtractedGarment, UserProfile, OutfitRecommendation.
app/paths.py	Filesystem	Resolves persistent data/ directories, SQLite DB paths, and image storage paths.
app/profile_flow.py	Wizard	7-step interactive /profile ConversationHandler capturing body frame & preferences.
app/style_matrix.py	Rule Engine	Deterministic color harmony wheel, silhouette proportion rules, offline fallback stylist.
app/stylist_graph.py	Agent Core	Stateful LangGraph orchestrator executing RAG context -> LLM stylist -> Critique loop.
app/weather.py	RAG Tool	Open-Meteo REST client retrieving real-time temperature, precipitation, and UV index.
app/web_search.py	RAG Tool	DuckDuckGo (ddgs) client retrieving live occasion-specific fashion trend snippets.
build_pdf.py	Docs Utility	Compiles comprehensive system_documentation.pdf engineering reference guide.
test_*.py	Verification	Automated test suite (intake, duplicate prevention, wardrobe CRUD, admin pool mode).

5. Tech Stack & Dependencies

Layer	Technology	Version	Purpose
Language	Python	>= 3.10	Core asynchronous runtime
Bot Framework	python-telegram-bot	>= 21.0	Async MTProto Telegram API wrapper
LLM / Vision	AWS Bedrock (Nova / Claude)	Converse API	Multimodal extraction & style reasoning
Agent Orchestration	LangGraph	>= 0.2.0	State machine with RAG & self-critique loop
Data Contracts	Pydantic	>= 2.5.0	Strict JSON schema validation and type safety
Database	SQLite 3	Built-in	Local relational storage with auto-migrations
Image Processing	Pillow (PIL)	>= 10.0.0	Resizing, 64-bit pHash, and labeled image badges
Web Trends RAG	duckduckgo-search / ddgs	>= 6.0.0	Live occasion & location fashion trend retrieval
Weather API	Open-Meteo API	REST v1	Keyless real-time weather & temperature forecasting

6. Database Architecture (SQLite)

The SQLite schema (data/wardrobe.db) provides normalized relational persistence with automatic column migration:

- garments: Stores individual clothing items: item_id, user_id, category, sub_category, color, accent_colors, silhouette_fit, fabric_weight, formality_tier, brand, style_tags, in_laundry, is_verified, created_at.
- garment_appearances: Tracks all photo appearances of an item, enabling multiple pictures (intake photo, OOTD snaps) to link to a single physical piece without duplicating storage.
- wear_history: Logs outfit wear events, timestamps, and occasions. Powering the 48-hour anti-repeat rotation cooldown and rejected-outfit blacklisting.
- user_profiles: Stores user gender frame, height, weight, body build, proportions (e.g. broad shoulders), preferred fits, and thermal preference (warm/cold).
- user_outfits: Saves verified favorite outfit combinations for user reference and style similarity RAG.

7. AI Models, Prompts & Contracts

The system defines strict JSON contracts across distinct prompt roles to guarantee schema compliance:

Prompt Role	Input Data	Output Contract	Responsibility
Vision Cataloger	JPEG bytes + caption	ExtractedGarment JSON	Extracts category, colors, fit, fabric weight, formality tier, brand.
Identity Decider	2 Garment dicts	Boolean Match JSON	Conservative match determining if two records represent the exact same piece.
Correction Refiner	Previous JSON + Text	Corrected JSON	Applies plain-language user modifications authoritatively.
Stylist Engine	Wardrobe + Weather + RAG	OutfitRecommendation JSON	Assembles harmonious multi-piece outfits with styling commentary.

8. Verification & Automated Test Suite

The test suite provides comprehensive coverage across all operational pathways:

- test_admin_pool.py: Verifies admin test pool isolation, markdown escaping, and wardrobe action keyboards.
- test_batch_and_wardrobe.py: Verifies multi-photo batch intake debouncing, 4-word title display caps, and category filtering.
- test_duplicate_detection.py: Verifies 64-bit pHash calculations, hue similarity filters, and dual-condition linking.
- test_intake_flow.py: Verifies HITL verification flow, single-item edits, and wardrobe navigation.

9. Environment Setup & Configuration

Step-by-step instructions to configure and execute the application:

```
# 1. Create and activate virtual environment
python3 -m venv .venv && source .venv/bin/activate

# 2. Install production dependencies
pip install -r requirements.txt

# 3. Configure environment secrets (.env)
cp .env.example .env
# Fill in: TELEGRAM_BOT_TOKEN, AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY

# 4. Launch bot application
python bot.py
```

10. Zero-Friction Judge Evaluation Guide

Judges can test the stylist immediately without needing to photograph 10+ clothing items:

- Step 1: Activate Test Pool: Send /admintest to the bot on Telegram and enter the demo password (demo123).
- Step 2: Inspect Wardrobe: Send /wardrobe to browse pre-seeded items across Tops, Bottoms, Footwear, and Outerwear.
- Step 3: Test Styling Engine: Send /style dinner date in Tokyo or /style casual rainy day to see live Weather RAG, DDGS web trend RAG, and Bedrock reasoning in action.
- Step 4: Exit Pool Mode: Send /adminlive to return to your private user wardrobe session.

11. VS Code to GitHub Upload Guide

Safe procedure to publish the project to GitHub without exposing secrets:

- 1. Verify .gitignore: Ensure .gitignore excludes .env, data/*.db, and data/images/* (run 'git status' to confirm).
- 2. Stage & Commit in VS Code: Open Source Control (Ctrl+Shift+G / Cmd+Shift+G) -> click '+' (Stage All) -> type commit message -> click Commit.
- 3. Publish Repository: Click 'Publish Branch' or 'Publish to GitHub' in VS Code, choose public/private, and push.